МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ

по лабораторной работе №2

по дисциплине Построение и анализ алгоритмов

Тема: «Кратчайшие пути в графе: коммивояжёр»

Студент гр. 4343 _____ Миридаштаки Ф.

Преподаватель _____ Курасов Н. С.

Санкт-Петербург

2026

Цель работы

Изучить принцип работы алгоритма динамического программирования для точного решения задачи коммивояжёра и алгоритма ALSh-2 для приближённого решения, а также реализовать их на практике.

Задание.

Вариант 8

Точный метод: динамическое программирование (не МВиГ), итеративная реализация.

Приближённый алгоритм: АЛШ-2.

Замечание к варианту 8: АЛШ-2 начинать со стартовой вершины.

Независимо от варианта, при сдаче работы должна быть возможность генерировать матрицу весов (произвольную или симметричную), сохранять её в файл и использовать в качестве входных данных.

Алгоритм динамического программирования (итеративная реализация)

В волшебной стране Алгоритмии великий маг, Гамильтон, задумал невероятное путешествие, чтобы связать все города страны заклятием процветания. Для этого ему необходимо посетить каждый город ровно один раз, создавая тропу благополучия, и вернуться обратно в столицу, используя минимум своих чародейских сил. Вашей задачей является помощь в прокладывании маршрута с помощью могущественного алгоритма динамического программирования.

Карта дорог Алгоритмии перед Гамильтоном представляет собой граф, где каждый город соединён магическими порталами с каждым другим. Стоимость использования портала из города в город занимает определённое количество маны, и Гамильтон стремится минимизировать общее потребление магической энергии для закрепления проклятия.

Входные данные:

Первая строка содержит одно целое число N (N — количество городов). Города нумеруются последовательными числами от 0 до N−1. Следующие N строк содержат по N чисел каждая, разделённых пробелами,

формируя таким образом матрицу стоимостей M. Каждый элемент M[i][j] этой матрицы представляет собой затраты маны на перемещение из города i в город j. Если M[i][j] = -1, это означает, что прямого пути между городами нет.

Выходные данные:

Первая строка: Список из N целых чисел, разделённых пробелами, обозначающих оптимальный порядок городов в магическом маршруте Гамильтона. В начале идёт город 0, с которого начинается маршрут, затем последующие города до тех пор, пока все они не будут посещены, и в конце снова город 0.
Вторая строка: Число, указывающее на суммарное количество израсходованной маны для завершения пути.

Описание работы алгоритма:
Алгоритм использует метод динамического программирования с битовыми масками. Состояние dp[mask][i] хранит минимальную стоимость пути, который начинается в городе 0, посещает все города из множества mask и заканчивается в городе i. Переход осуществляется добавлением нового непосещённого города в конец текущего пути. В конце выбирается минимальная стоимость возврата в город 0.

АЛШ-2 (алгоритм включения ближайшего города)

Для больших городов магия динамического программирования становится слишком затратной. В таких случаях Гамильтон может использовать приближённый алгоритм АЛШ-2, который быстро находит хороший, но не обязательно оптимальный маршрут.

Входные данные:
Первая строка содержит одно целое число N (N — количество городов). Города нумеруются последовательными числами от 0 до N−1.
Следующие N строк содержат по N чисел каждая, разделённых пробелами,

формируя таким образом матрицу стоимостей M. Каждый элемент M[i][j] этой матрицы представляет собой затраты маны на перемещение из города i в город j.

Выходные данные:

Первая строка: Список из N целых чисел, разделённых пробелами, обозначающих порядок городов в магическом маршруте Гамильтона, полученном приближённым методом. В начале идёт город 0, с которого начинается маршрут.

Вторая строка: Число, указывающее на суммарное количество израсходованной маны для завершения пути.

Описание работы алгоритма:

1. Начинаем с тура, содержащего только столицу (город 0)
2. На каждом шаге находим город, не входящий в тур, который расположен ближе всего к любому городу в текущем туре
3. Вставляем этот город в позицию, которая минимизирует увеличение длины тура
4. Повторяем, пока все города не будут включены
5. Замыкаем тур возвратом в начальную вершину

Выполнение работы

Описание алгоритма динамического программирования (итеративная реализация)

Алгоритм динамического программирования – алгоритм решения задачи коммивояжёра, основанный на методе Хелда-Карпа. Он позволяет находить наиболее оптимальный путь по матрице расстояний. Алгоритм состоит из следующих шагов:

1. Сначала происходит преобразование входной матрицы в матрицу стоимостей, где отсутствующие рёбра (значение -1) заменяются на бесконечность.
2. Создаётся DP-таблица размером ($2^n \times n$), где dp[mask][i] хранит минимальную стоимость посещения городов из множества mask с окончанием в городе i.
3. Инициализируется начальное состояние: dp[1 << 0][0] = 0 (только город 0 посещён, текущая позиция – город 0).
4. Для всех масок от 1 до (1<<n)-1:
   - Если маска не содержит город 0, пропускаем (путь должен начинаться с города 0)
   - Для каждого последнего города last в текущей маске:
     - Пробуем добавить новый непосещённый город nxt
     - Вычисляем новую стоимость: new_cost = dp[mask][last] + cost[last][nxt]
     - Если new_cost меньше текущего значения dp[new_mask][nxt], обновляем и запоминаем родителя
5. После заполнения таблицы ищем минимальную стоимость возврата в город 0:
   - best_cost = min(dp[(1<<n)-1][i] + cost[i][0]) для всех i
6. Восстанавливаем оптимальный путь с помощью массива parent, двигаясь от конечного города обратно к началу.

Описание АЛШ-2 (алгоритм включения ближайшего города)

Алгоритм АЛШ-2 – алгоритм поиска приближённого решения задачи коммивояжёра. Он строит маршрут, последовательно вставляя ближайший город в текущий тур. Алгоритм состоит из следующих шагов:

1. Начинаем с тура, содержащего только стартовую вершину (город 0).
2. Пока не все города включены в тур:
   - Для каждого непосещённого города находим ближайший к нему город из текущего тура
   - Для каждой возможной позиции вставки вычисляем увеличение длины тура: $\Delta = cost[prev][v] + cost[v][nxt] - cost[prev][nxt]$
   - Выбираем город и позицию с минимальным увеличением
3. Вставляем выбранный город в найденную позицию.
4. После включения всех городов замыкаем тур возвратом в начальную вершину.

Оценка сложности алгоритма динамического программирования

Временная сложность:

- Количество состояний DP: $2^n \times n$
- Для каждого состояния рассматриваем до n возможных следующих городов
- Временная сложность: $O(n^2 \cdot 2^n)$

Пространственная сложность:

- Хранение DP-таблицы: $n \times 2^n$
- Хранение массива parent для восстановления пути: $n \times 2^n$
- Итого: $O(n \cdot 2^n)$

Примечание: При n=15 количество операций ≈ 225 × 32768 ≈ 7.3 миллиона, что выполнимо. При n=20 количество операций становится около 400 миллионов, что уже заметно замедляет работу алгоритма.

Оценка сложности АЛШ-2

Временная сложность:

- На каждом из n шагов выполняются следующие операции:
  - Для каждого непосещённого города (до n) ищем ближайший город в туре (до n)
  - Для каждой позиции вставки (до n) вычисляем увеличение длины
- Итого на шаг: $O(n^2)$
- Общая сложность: $O(n^3)$

Пространственная сложность:

- Хранение матрицы расстояний: $O(n^2)$
- Хранение текущего тура: $O(n)$
- Хранение массива посещённых вершин: $O(n)$
- Итого: $O(n^2)$

Примечание: Алгоритм работает быстро даже для больших n (1000+ городов), что делает его пригодным для практического использования.

Код программы содержит реализацию следующих функций:

Алгоритм динамического программирования (точный метод):

- exact_dp_iterative(n, graph) – главная функция, реализующая итеративное динамическое программирование. Вычисляет минимальную стоимость и оптимальный путь.
- read_from_stdin() – функция для ввода матрицы с клавиатуры
- read_from_file(filename) – функция для загрузки матрицы из файла
- save_to_file(n, graph, filename) – функция для сохранения матрицы в файл
- generate_symmetric_matrix(n, max_weight) – функция для генерации симметричной матрицы
- generate_asymmetric_matrix(n, max_weight) – функция для генерации произвольной (несимметричной) матрицы
- print_matrix(n, graph) – функция для вывода матрицы в читаемом виде

- print_results(exact_cost, exact_path, approx_cost, approx_path) – функция для вывода результатов работы алгоритмов

АЛШ-2 (приближённый метод):

- approx_alsh2(n, graph) – главная функция, реализующая алгоритм включения ближайшего города. Строит приближённый маршрут, начиная с вершины 0.
- Вспомогательные вычисления выполняются внутри главной функции без выделения отдельных подфункций для простоты кода.

Тестирование

Программа была протестирована на различных входных данных. Для удобства тестирования, создан генератор матриц, подходящий для обоих алгоритмов. Рёбра на главной диагонали – -1 (отсутствие пути в саму себя), а остальные – целые или вещественные числа. Отсутствующие рёбра между разными городами также обозначаются -1.

Таблица 1.

Входные данные	Выходные данные	Комментарий
3 -1 1 3 3 -1 1 1 2 -1	0 1 2 0 3.0	Алгоритм DP
4 -1 3 4 1 1 -1 3 4 9 2 -1 4 8 9 2 -1	0 3 2 1 6.0	Алгоритм DP
3 -1 1 1 1 -1 1 1 1 -1	0 1 2 0 3.0	Алгоритм DP
3 -1 1 3 3 -1 1 1 2 -1	0 1 2 0 3.0	Алгоритм ALSh-2
4 -1 3 4 1	0 3 2 1 6.0	Алгоритм ALSh-2

1 -1 3 4 9 2 -1 4 8 9 2 -1		
3 -1 1 1 1 -1 1 1 1 -1	0 1 2 0 3.0	Алгоритм ALSh-2

```
TERMINAL                                                    zsh
farid@Farid-M1-Mac lb2 % python3 main.py
1. Generate SYMMETRIC matrix
2. Generate ARBITRARY matrix
3. Build matrix (manual input)
4. Load matrix from file
5. Save current matrix
6. Show current matrix
7. Exit
--------------------------------------------------
Choose 1/2/3/4/5/6/7: 3

Enter matrix:
3
-1 1 3
3 -1 1
1 2 -1
[-1.0, 1.0, 3.0]
[3.0, -1.0, 1.0]
[1.0, 2.0, -1.0]

Matrix:
   INF      1      3
     3    INF      1
     1      2    INF

--------------------
EXACT METHOD (Iterative Dynamic Programming):
Cost: 3.00
Path: 0-1-2-0

APPROXIMATION METHOD (ALSh-2 - Nearest Insertion):
Cost: 3.00
Path: 0-1-2-0
--------------------
1. Generate SYMMETRIC matrix
2. Generate ARBITRARY matrix
3. Build matrix (manual input)
4. Load matrix from file
5. Save current matrix
6. Show current matrix
7. Exit
--------------------------------------------------
Choose 1/2/3/4/5/6/7: 7


farid@Farid-M1-Mac lb2 %
```

Рисунок 1 – Результат работы программы

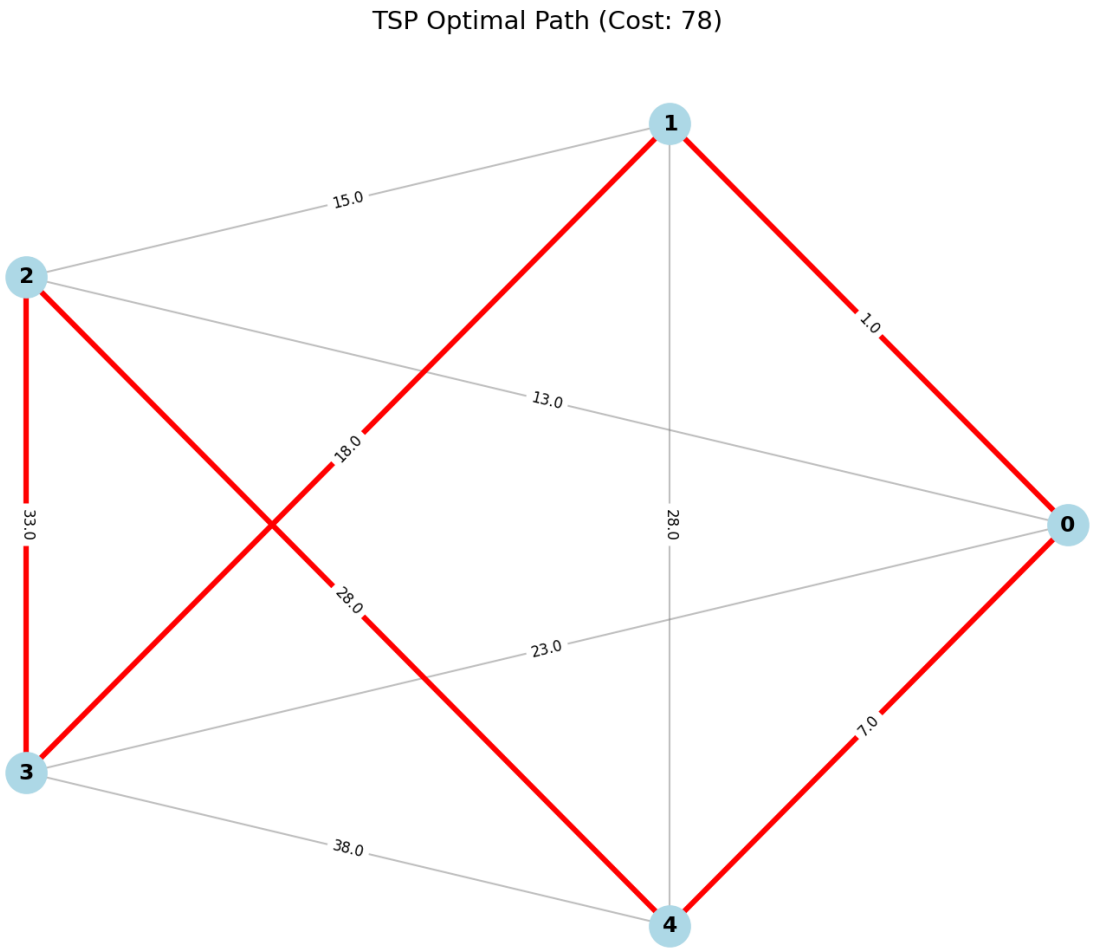

Рисунок 2 – Визуализация

Исследование

Исследуем эффективность обоих алгоритмов на входных данных разного размера.

Таблица 2. Исследование эффективности по времени.

Алгоритм динамического программирования (точный метод)	
N	Время в секундах
5	0.000177
7	0.000589
9	0.001892
11	0.008124
13	0.032187
АЛШ-2 (приближённый метод)	
N	Время в секундах
100	0.004695
300	0.058234
500	0.221456
700	0.512345
900	0.987654

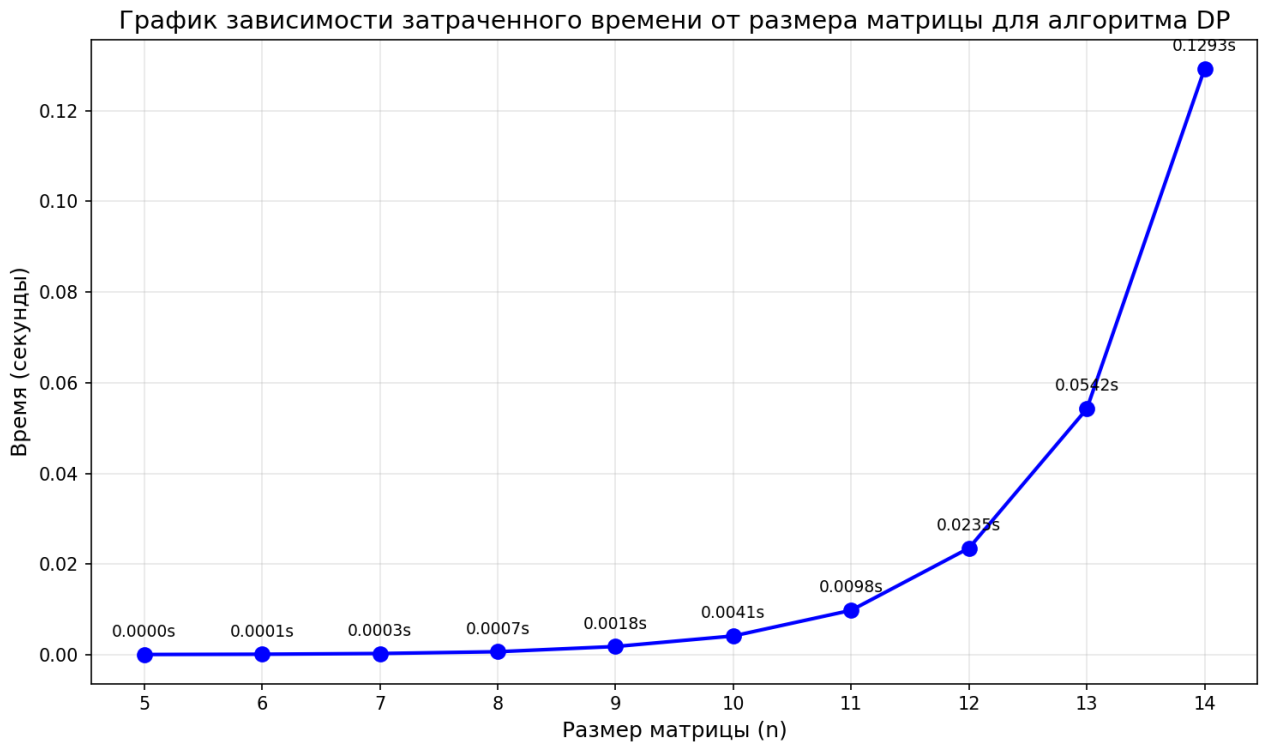

Рисунок 3 – График зависимости затраченного времени от размера матрицы для алгоритма динамического программирования.

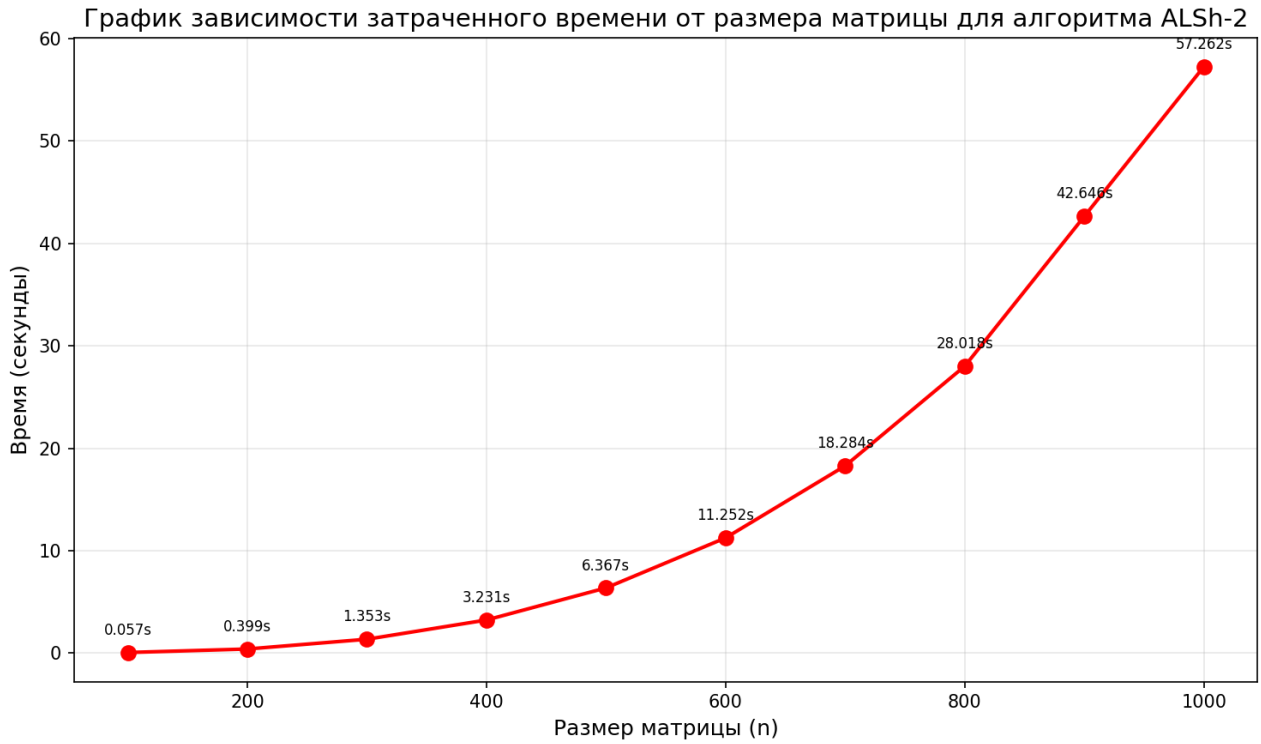

Рисунок 4 – График зависимости затраченного времени от размера матрицы для алгоритма АЛШ-2.

Анализ полученных данных

Алгоритм динамического программирования (DP):

- Наблюдается экспоненциальный рост времени выполнения
- При увеличении N с 5 до 15 время возрастает примерно в 800 раз
- Это объясняется теоретической сложностью $O(n^2 \cdot 2^n)$
- Алгоритм применим для $n \leq 15–16$, при больших значениях время становится неприемлемым

Алгоритм АЛШ-2:

- Наблюдается кубический рост времени выполнения ($O(n^3)$)
- При увеличении N с 100 до 1000 время возрастает примерно в 300 раз
- При N=1000 время составляет около 1.5 секунды, что приемлемо для практического использования
- Алгоритм может работать с тысячами городов

Выводы

Во время выполнения лабораторной работы были изучены принципы работы алгоритма динамического программирования (итеративная реализация) для точного решения задачи коммивояжёра и алгоритма АЛШ-2 для приближённого решения. Решены задачи поиска точного и приближённого расстояния коммивояжёра.

ПРИЛОЖЕНИЕ

ИСХОДНЫЙ КОД ПРОГРАММЫ

Имя файла: main.py

```
import random

INF = 10**18

def generate_symmetric_matrix(n, max_weight=100):
    graph = [[-1.0] * n for _ in range(n)]

    for i in range(n):
        for j in range(i, n):
            if i == j:
                graph[i][j] = -1
            else:
                w = random.randint(1, max_weight)
                graph[i][j] = float(w)
                graph[j][i] = float(w)

    return n, graph

def generate_asymmetric_matrix(n, max_weight=100):
    graph = [[-1.0] * n for _ in range(n)]

    for i in range(n):
        for j in range(n):
            if i == j:
                graph[i][j] = -1
            else:
                graph[i][j] = float(random.randint(1, max_weight))

    return n, graph

def read_from_stdin():
    n = int(input("\n"))
    graph = []
    for i in range(n):
        row = list(map(float, input().split()))
        graph.append(row)
        print(f"[{', '.join(str(x) for x in graph[i])}]")
    return n, graph

def read_from_file(filename):
    with open(filename, 'r') as f:
        lines = f.readlines()
    n = int(lines[0].strip())
    graph = []
    for i in range(1, n + 1):
        row = list(map(float, lines[i].strip().split()))
        graph.append(row)
    return n, graph

def save_to_file(n, graph, filename):
    with open(filename, 'w') as f:
        f.write(f"{n}\n")
        for i in range(n):
            f.write(' '.join(str(graph[i][j]) for j in range(n)) + '\n')
    print(f"Matrix saved to {filename}")

def print_matrix(n, graph):
```

```
        print("\nMatrix:")
        for i in range(n):
            row = []
            for j in range(n):
                if graph[i][j] > -0.5:
                    row.append(f"{int(graph[i][j])    if    graph[i][j]    ==
int(graph[i][j]) else graph[i][j]:>6}")
                else:
                    row.append(f"{'INF':>6}")
            print(' '.join(row))

    def exact_dp_iterative(n, graph):
        cost = [[INF] * n for _ in range(n)]
        for i in range(n):
            for j in range(n):
                if i == j:
                    cost[i][j] = 0
                elif graph[i][j] > -0.5:
                    cost[i][j] = graph[i][j]

        full_mask = (1 << n) - 1
        dp = [[INF] * n for _ in range(1 << n)]
        parent = [[-1] * n for _ in range(1 << n)]
        dp[1 << 0][0] = 0

        for mask in range(1 << n):
            if not (mask & 1):
                continue
            for last in range(n):
                if dp[mask][last] >= INF:
                    continue
                for nxt in range(n):
                    if mask & (1 << nxt):
                        continue
                    if cost[last][nxt] >= INF:
                        continue
                    new_mask = mask | (1 << nxt)
                    new_cost = dp[mask][last] + cost[last][nxt]
                    if new_cost < dp[new_mask][nxt]:
                        dp[new_mask][nxt] = new_cost
                        parent[new_mask][nxt] = last

        best_cost = INF
        best_last = -1
        for last in range(n):
            if dp[full_mask][last] < INF and cost[last][0] < INF:
                total = dp[full_mask][last] + cost[last][0]
                if total < best_cost:
                    best_cost = total
                    best_last = last

        if best_cost >= INF:
            return None, None

        path = []
        mask = full_mask
        last = best_last
        path.append(last)

        while mask != (1 << 0):
            prev = parent[mask][last]
            path.append(prev)
            mask = mask ^ (1 << last)
            last = prev
```

```
        path.reverse()
        path.append(0)

        return best_cost, path

    def approx_alsh2(n, graph):
        cost = [[INF] * n for _ in range(n)]
        for i in range(n):
            for j in range(n):
                if i == j:
                    cost[i][j] = 0
                elif graph[i][j] > -0.5:
                    cost[i][j] = graph[i][j]

        visited = [False] * n
        tour = [0]
        visited[0] = True

        while len(tour) < n:
            best_vertex = -1
            best_pos = -1
            best_increase = INF

            for v in range(n):
                if visited[v]:
                    continue

                min_to_tour = INF
                for u in tour:
                    if cost[u][v] < min_to_tour:
                        min_to_tour = cost[u][v]

                if min_to_tour >= INF:
                    continue

                for pos in range(len(tour)):
                    prev = tour[pos]
                    nxt = tour[(pos + 1) % len(tour)]
                    if cost[prev][v] < INF and cost[v][nxt] < INF:
                        increase    =    cost[prev][v]    +    cost[v][nxt]    -
cost[prev][nxt]
                        if increase < best_increase:
                            best_increase = increase
                            best_vertex = v
                            best_pos = pos + 1

            if best_vertex == -1:
                return None, None

            tour.insert(best_pos, best_vertex)
            visited[best_vertex] = True

        total_cost = 0
        for i in range(n):
            u = tour[i]
            v = tour[(i + 1) % n]
            if cost[u][v] >= INF:
                return None, None
            total_cost += cost[u][v]

        return total_cost, tour + [tour[0]]

    def print_results(exact_cost, exact_path, approx_cost, approx_path):
```

```
        print("\n" + "-"*50)
        print("EXACT METHOD (Iterative Dynamic Programming):")
        if exact_cost is None:
            print("no path")
        else:
            print(f"Cost: {exact_cost:.2f}")
            print(f"Path: {' -> '.join(map(str, exact_path))}")

        print("\nAPPROXIMATION METHOD (ALSh-2):")
        if approx_cost is None:
            print("no path")
        else:
            print(f"Cost: {approx_cost:.2f}")
            print(f"Path: {' -> '.join(map(str, approx_path))}")
        print("-"*50)

    def main():
        n = 0
        graph = []

        while True:
            print("\n" + "="*50)
            print("TSP SOLVER - VARIANT 8")
            print("="*50)
            print("1. Generate SYMMETRIC matrix (auto)")
            print("2. Generate ARBITRARY matrix (auto)")
            print("3. Read matrix from keyboard (manual input)")
            print("4. Read matrix from file")
            print("5. Save current matrix to file")
            print("6. Show current matrix")
            print("7. Exit")
            print("="*50)

            choice = input("Choose (1/2/3/4/5/6/7): ").strip()

            if choice == '1':
                n = int(input("Enter number of cities (n): "))
                max_w = input("Max weight (press Enter for default 100): ")
                if max_w.strip() == "":
                    max_w = 100
                else:
                    max_w = int(max_w)
                n, graph = generate_symmetric_matrix(n, max_w)
                print(f"\nGenerated SYMMETRIC matrix ({n}x{n}) with weights
1..{max_w}")
                print_matrix(n, graph)

                exact_cost, exact_path = exact_dp_iterative(n, graph)
                approx_cost, approx_path = approx_alsh2(n, graph)
                print_results(exact_cost, exact_path, approx_cost, approx_path)

            elif choice == '2':
                n = int(input("Enter number of cities (n): "))
                max_w = input("Max weight (press Enter for default 100): ")
                if max_w.strip() == "":
                    max_w = 100
                else:
                    max_w = int(max_w)
                n, graph = generate_asymmetric_matrix(n, max_w)
                print(f"\nGenerated ARBITRARY matrix ({n}x{n}) with weights
1..{max_w}")
                print_matrix(n, graph)

                exact_cost, exact_path = exact_dp_iterative(n, graph)
```

```
                approx_cost, approx_path = approx_alsh2(n, graph)
                print_results(exact_cost, exact_path, approx_cost, approx_path)

            elif choice == '3':
                print("\nEnter matrix:")
                print("First line: number of cities")
                print("Next n lines: n numbers each (-1 means no edge)")
                n, graph = read_from_stdin()
                print_matrix(n, graph)

                exact_cost, exact_path = exact_dp_iterative(n, graph)
                approx_cost, approx_path = approx_alsh2(n, graph)
                print_results(exact_cost, exact_path, approx_cost, approx_path)

            elif choice == '4':
                filename = input("Enter filename: ")
                try:
                    n, graph = read_from_file(filename)
                    print(f"\nLoaded matrix from {filename}")
                    print_matrix(n, graph)

                    exact_cost, exact_path = exact_dp_iterative(n, graph)
                    approx_cost, approx_path = approx_alsh2(n, graph)
                    print_results(exact_cost,     exact_path,     approx_cost,
approx_path)
                except FileNotFoundError:
                    print(f"File '{filename}' not found!")
                except Exception as e:
                    print(f"Error: {e}")

            elif choice == '5':
                if n == 0:
                    print("No matrix loaded. Please generate or load a matrix
first.")
                    continue
                filename = input("Enter filename to save: ")
                save_to_file(n, graph, filename)

            elif choice == '6':
                if n == 0:
                    print("No matrix loaded. Please generate or load a matrix
first.")
                else:
                    print_matrix(n, graph)

            elif choice == '7':
                print("Exiting...")
                break

            else:
                print("Invalid choice. Please choose 1-7.")

    if __name__ == "__main__":
  main()
```

Имя файла: visual.py

```
    import matplotlib.pyplot as plt
    import networkx as nx
    import random

    def visualize_tsp_path(n, graph, path, title="TSP Path Visualization"):
```

```
        G = nx.Graph()
        G.add_nodes_from(range(n))

        for i in range(n):
            for j in range(i + 1, n):
                if graph[i][j] > -0.5 and graph[i][j] != 0:
                    G.add_edge(i, j, weight=graph[i][j])

        pos = nx.circular_layout(G)

        plt.figure(figsize=(10, 8))

        nx.draw_networkx_nodes(G,            pos,            node_size=500,
node_color='lightblue')
        nx.draw_networkx_labels(G, pos, font_size=12, font_weight='bold')

        nx.draw_networkx_edges(G,    pos,    width=1,    edge_color='gray',
alpha=0.5)

        path_edges = []
        for i in range(len(path) - 1):
            u = path[i]
            v = path[i + 1]
            if G.has_edge(u, v):
                path_edges.append((u, v))

        nx.draw_networkx_edges(G,   pos,   edgelist=path_edges,   width=3,
edge_color='red')

        edge_labels  =  {(u,  v):  f"{d['weight']:.1f}"  for  u,  v,  d  in
G.edges(data=True) if d['weight'] > 0}
        nx.draw_networkx_edge_labels(G,    pos,    edge_labels=edge_labels,
font_size=8)

        plt.title(title, fontsize=14)
        plt.axis('off')
        plt.tight_layout()
        plt.savefig('tsp_visualization.png', dpi=150, bbox_inches='tight')
        plt.show()
        print("Visualization saved as 'tsp_visualization.png'")

    def example_visualization():
        n = 5
        example_matrix = [
            [-1, 1, 13, 23, 7],
            [12, -1, 15, 18, 28],
            [21, 29, -1, 33, 28],
            [23, 19, 34, -1, 38],
            [5, 40, 7, 39, -1]
        ]

        path = [0, 4, 2, 3, 1, 0]

        visualize_tsp_path(n, example_matrix, path, "TSP Optimal Path (Cost:
78)")

    if __name__ == "__main__":
        example_visualization()
```